



MEMORIA PRACTICA 1

DIVIDE Y VENCERAS



Profesor: Jesús Sánchez Cuadrado

Alumnas:

María Ballester Martínez (m.ballestermartinez@um.es) 48853989B

Lucía Olmos Martínez (lucia.olmosm@um.es) 49339235A

Grupo: 1.1

Ejercicio a resolver: 10

ÍNDICE

1. Enunciado.....	2
2. Pseudocódigo y explicación del algoritmo.....	2
3. Estudio teórico del tiempo de ejecución del algoritmo	10
4. Programación del algoritmo.....	11
5. Validación del algoritmo	19
6. Estudio experimental del tiempo de ejecución	20
7. Contraste del estudio teórico y el experimental	22
8. Conclusión	24

1. Enunciado

Dada una cadena C con n caracteres, hay que encontrar la subcadena más larga compuesta por parejas consecutivas de consonante-vocal. devolver el índice p de comienzo de la solución y la subcadena solución.

Por ejemplo, si C = abccabeaddedebedae

la solución es dedade en la posición 10.

En caso de empate, será válida cualquiera de las soluciones óptimas.

2. Pseudocódigo y explicación del algoritmo

constante int N = 170000

array caracteres A

ofstream cMejor, cPeor, cTodos

constante entero minimo = 10000

constante entero maximo = 210000

La función esVocal indica si el carácter que se encuentra en una cierta posición “ i ” del array es una vocal o no.

función esVocal (entero i) --> booleano

según A[i]

caso 'a', 'e', 'i', 'o', 'u':

devolver verdadero

caso contrario:

devolver falso

A la función solucionDirecta le llega la posición de dos letras en el array y devuelve esas posiciones si es una pareja de consonante-vocal, ya que encaja en el patrón; si no, devuelve 0,0 que significa que no cuadra con el patrón buscado.

función soloucionDirecta (entero p, q) --> par<entero,entero>

si no esVocal(p) Y esVocal(q)

devolver p,q

sino

devolver 0,0

La función dividir, como bien indica el nombre, divide el rango dado en dos partes para estudiarlas como subproblemas.

función dividir (entero p, q) --> entero

devolver entero((q+p)/2)

La función cadMLarga devuelve cuál es la cadena más larga de las dos que se le pasan como parámetro. Si son del mismo tamaño devuelve la primera.

función cadMLarga (par<entero,entero> s1, s2) --> par<entero,entero>

si (s1.segundo - s1.primerio) >= (s2.segundo - s2.primerio)

devolver s1

sino

devolver s2

La función extenderDer va comprobando que los dos caracteres más a la derecha del final de la solución actual cumplan el patrón consonante-vocal y así ir extendiendo la solución por la derecha.

función extenderDer (par<entero,entero> s1) --> par<entero,entero>

entero x = s1.segundo

mientras (x+2 <= tamaño(A)-1) Y solucionDirecta(x+1, x+2) no es par{0,0}

x += 2

devolver s1.primerio,x

La función extenderIzq va comprobando que los dos caracteres que se encuentran a la izquierda del principio de la solución actual sigan el patrón consonante-vocal, pudiendo extender por la izquierda.

función extenderIzq (par<entero,entero> s2) --> par<entero,entero>

entero x = s2.primerio

mientras (x-2 >= 0) Y solucionDirecta(x-2, x-1) no es par{0,0}

x -= 2

devolver x,s2.segundo

La función `cadFrontera` toma la solución que se encuentra en la frontera de dos subproblemas contiguos, extendiendo por ambos lados para conseguir la cadena más larga posible.

función `cadFrontera` (entero p, q, m) --> par<entero,entero>

par<entero,entero> $s3 = m, m+1$

$s3 = \text{extenderIzq}(s3)$

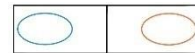
$s3 = \text{extenderDer}(s3)$

devolver $s3$

La función `combinar` recibe dos pares de números, dos enteros que marcan el rango del array a estudiar y el resultado de dividir ese rango.

Esta función estudia, con cada `if`, los diferentes casos que se pueden dar:

1. Si no hay continuidad en la frontera devuelve la cadena más larga de las dos comparadas.
2. Si hay continuidad en la frontera:
 - a. Puede existir el caso en el que tanto $s1$ como $s2$ toquen la frontera.
 - b. Otro caso es si $s1$ toca la frontera y $s2$ no.
 - c. Además, existe el caso de que $s2$ toque la frontera y $s1$ no.
 - d. Por último, si la `solucionDirecta` de la frontera es distinto de 0,0.



función `combinar` (par<entero,entero> $s1, s2$; entero p, q, m) --> par<entero,entero>

si `solucionDirecta`($m, m+1$) es par{0,0}

devolver `cadMLarga`($s1, s2$)

sino

si $s1.segundo$ es m

si $s2.primer$ es $m+1$

devolver $s1.primer, s2.segundo$

sino

$s1 = \text{extenderDer}(s1)$

devolver `cadMLarga`($s1, s2$)

sino

si $s2.primer$ es $m+1$

$s2 = \text{extenderIzq}(s2)$

devolver `cadMLarga`($s1, s2$)

```

sino
    par<entero,entero> s3 = cadFrontera(p, q, m)
    par<entero,entero> s4 = cadMLarga(s1, s2)
    devolver cadMLarga(s3, s4)

devolver 0,0

```

La función recursiva DyV comprueba tres posibles casos que pueden suceder: que el array sea de tamaño 1 o 0 (no hay solución), que el array sea de tamaño 2 (va a solución directa), y que el array sea de tamaño suficiente como para dividirlo en subproblemas. En este caso se realizará este mismo proceso para cada subproblema y se irán combinando para encontrar la solución más larga posible.

```

función DyV (entero p, q) --> par<entero,entero>
    si p es q
        devolver 0,0
    sino si q-p es 1
        devolver solucionDirecta(p, q)
    sino
        entero m = dividir(p, q)
        par<entero,entero> s1 = DyV(p, m)
        par<entero,entero> s2 = DyV(m+1, q)
        devolver combinar(s1, s2, p, q, m)

```

La función **generadorEntrada** genera un array de tamaño N con caracteres aleatorios.

```

función generadorEntrada (entero n) --> array caracteres
    array caracteres cadena[n]
    para i = 0 ... n
        cadena[i] = carácter(97 + rand() % (122 - 97 + 1))
    devolver cadena

```

La función **generadorEntradaPeor** genera un array de tamaño N con el peor caso de entrada.

```

función generadorEntradaPeor (entero n) --> array caracteres
    array caracteres cadena[n]
    para i = 0 ... n

```

```

        si i%2 es 0
            cadena[i] = 'b'
        sino
            cadena[i] = 'a'

    devolver cadena

```

La función `generadorEntradaMejor` genera un array de tamaño N con el mejor caso de entrada.

```

función generadorEntradaMejor (entero n) --> array caracteres
    array caracteres cadena[n]
    para i = 0 ... n
        cadena[i] = 'a'
    devolver cadena

```

La función `imprimirSol` imprime en la terminal cuál es la solución.

```

función imprimirSol (par<entero,entero> solucion)
    si solucion es par{0,0}
        imprimir "No hay cadenas consonante-vocal de ninguna longitud"
    sino
        imprimir "La solución es "
        para i = solucion.primerio ... solucion.segundo
            imprimir A[i]
        imprimir " en la posición " solucion.first + 1
    }
}

```

La función `imprimirArray` imprime en la terminal el array.

```

función imprimirArray()
    para i = 0...tamaño(A)
        imprimir A[i]

```

La función obtenerTiempo calcula el tiempo que tarda en ejecutarse cada caso.

función obtenerTiempo(entero p, q) --> String

```
    estructura timeval t1, t2
    decimal tiempo
    gettimeofday(&t1, NULL)
    para<entero,entero> solucion = DyV(p, q)
    gettimeofday(&t2, NULL)
    tiempo = (t2.tv_sec - t1.tv_sec)*1000 + (t2.tv_usec - t1.tv_usec)/1000.0
    string t = ""
    string cadenaTiempo = to_string(tiempo)
    para h = 0 ... cadenaTiempo.length()
        si cadenaTiempo[h] no es '.'
            t = t + cadenaTiempo[h]
        sino t = t + ','
    devolver t
```

La función archivosCSV genera archivos CSV con los datos del tiempo generados por la función anterior.

función archivosCSV()

```
    cTodos.open("todos.csv")
    cPeor.open("peor.csv")
    cMejor.open("mejor.csv")
    entero p, q, m
    cMejor << "n;tm(n)" << endl
    cPeor << "n;tM(n)" << endl
    cTodos << "n;tM(n);tm(n);t"endl
    para n = minimo ... maximo
        p = 0
        q = n-1
        A = generadorEntradaPeor(n)
        string tM = obtenerTiempo(p, q)
        A = generadorEntradaMejor(n)
```



```
string tm = obtenerTiempo(p, q)
```

```
cTodos << n << ";" << tM << ";" << tm << endl
```

```
cMejor << n << ";" << tm << endl
```

```
cPeor << n << ";" << tM << endl
```

```
función main ()
```

```
    srand(time(NULL))
```

```
    imprimir "¿Qué desea hacer? (introduzca el número de la opción)"
```

```
    imprimir "1 – Analizar un número a indicar de arrays"
```

```
    imprimir "2 – Producir datos sobre el tiempo que se guardarán en un archivo .csv"
```

```
    imprimir "3 – Ayuda"
```

```
    entero opción
```

```
    leer opción
```

```
    si opcion es 1
```

```
        entero c
```

```
        imprimir "Introduzca el número de cadenas que quiere analizar"
```

```
        leer c
```

```
        entero array
```

```
        imprimir "¿Quiere saber cuáles son los arrays? (1 = SI; 0 = NO)"
```

```
        leer array
```

```
        para i = 0 ... c
```

```
            imprimir i+1 "-"
```

```
            A = generadorEntrada(N)
```

```
            si array es 1
```

```
                imprimirArray()
```

```
            entero p = 0
```

```
            entero q = tamaño(A)-1
```

```
            par<entero,entero> solución = DyV(p, q)
```

```
            imprimirSol(solucion)
```

sino si opción es 2

archivosCSV()

sino

imprimir "1 - Se generará de manera aleatoria un array de caracteres (alfabeto únicamente) que se analizará a través de un algoritmo basado en \"Divide y Vencerás\" para comprobar cuál es la subcadena más larga que cumpla con el patrón consonante-vocal. Usted determinará el número de cadenas a analizar."

imprimir "2 - Se hará un estudio del tiempo sobre dos tipos de entradas: el peor caso y el mejor. Insertará los datos en archivos .csv, es decir, en hojas de cálculo."

devolver 0

3. Estudio teórico del tiempo de ejecución del algoritmo

$$\left. \begin{array}{l} \text{seleccionarDirecta } t(n) = 2 \rightarrow \boxed{O(2)} \\ \text{dividir } t(n) = 1 \rightarrow \boxed{O(1)} \\ \text{codificarLarga } t(n) = 2 \rightarrow \boxed{O(2)} \end{array} \right\} \text{ No hay ni mejor ni peor caso}$$

$$\begin{aligned} \text{extenderDer } t_m(n) &= 1 + 1 + 1 = 3 \rightarrow \boxed{\Omega(3)} \\ t_u(n) &= 1 + \sum_{i=1}^{\log_2(n)} 2 + 1 + 1 = 3 + 2\log_2(n) \rightarrow \boxed{O(\log(n))} \end{aligned}$$

$$\begin{aligned} \text{extenderIzq } t_m(n) &= 1 + 1 + 1 = 3 \rightarrow \boxed{\Omega(3)} \\ t_u(n) &= 1 + \sum_{i=1}^{\log_2(n)} 2 + 1 + 1 = 3 + 2\log_2(n) \rightarrow \boxed{O(\log(n))} \end{aligned}$$

$$\begin{aligned} \text{codFrontera } t_m(n) &= 1 + t_m(n) \text{ extenderDer} + t_m(n) \text{ extenderIzq} + 1 = 2 + 3 + 3 = 8 \rightarrow \boxed{\Omega(8)} \\ t_u(n) &= 1 + t_u(n) \text{ extenderDer} + t_u(n) \text{ extenderIzq} + 1 = 2 + 2\log_2(n) \rightarrow \boxed{O(\log(n))} \end{aligned}$$

$$\begin{aligned} \text{combinar } t_m(n) &= 1 + t(n) \text{ codificarLarga} = 1 + 2 \cdot 3 \rightarrow \boxed{\Omega(3)} \\ t_u(n) &= 4 + t_u(n) \text{ codFrontera} + t(n) \text{ codificarLarga} + t(n) \text{ codificarLarga} = 4 + 2\log_2(n) + 2 + 2 \cdot 8 + 2\log_2(n) \rightarrow \boxed{O(\log(n))} \end{aligned}$$

$$\begin{aligned} D_0V \quad t_m(n) &= 2 + t(n) \text{ dividir} + 2t(n/2) + t_m(n) \text{ combinar} = 2t(n/2) + 6 \\ \text{Por el teorema maestro } T(n) &= a + \left(\frac{n}{b}\right) + O(n^k), \text{ donde en nuestro caso } a=2, b=2 \text{ y } k=0 \\ \text{por lo que } a &> b^k \rightarrow 2 > 2^0, \text{ entonces } O(n^{\log_2(2)}) = \boxed{O(n)} \\ t_u(n) &= 2 + t(n) \text{ dividir} + 2t(n/2) + t_u(n) \text{ combinar} = 2 + 1 + 2t(n/2) + \log_2(n) = 2t(n/2) + \log_2(n) + 3 \\ \text{Por el teorema maestro } T(n) &= a + \left(\frac{n}{b}\right) + O(n^k), \text{ donde en nuestro caso } a=2, b=2 \text{ y } k=1 \\ \text{por lo que } a &= b^k = 2 = 2^1, \text{ entonces } \boxed{O(n \cdot \log(n))} \end{aligned}$$

4. Programación del algoritmo

Debíamos realizar un programa con el algoritmo de Divide y Vencerás con el objetivo de encontrar, en un array de caracteres, la cadena más larga formada por el patrón consonante-vocal.

Para ello hemos creado una función DyV recursiva, la principal para resolver el problema, que se ocupará de llamar a funciones auxiliares que ayuden a encontrar una solución:

```
pair<int,int> DyV(int p, int q) {
    if (p == q)
        return {0,0};
    else if (q - p == 1)
        return solucionDirecta(p,q);
    else {

        int m = dividir(p,q);
        pair<int,int>s1 = DyV(p,m);
        pair<int,int>s2 = DyV(m+1,q);
        return combinar(s1,s2,p,q,m);
    }
}
```

“DyV” comprobará en primer lugar los casos base. En este caso encontramos dos posibilidades:

- Que el array sea de tamaño 1: no habrá posibilidad de encontrar una cadena válida, ya que solo habrá un carácter.
- Que el array sea de tamaño 2: es posible que haya una cadena válida al encontrar dos caracteres en el array. Llamará a “solucionDirecta” para comprobar esto:

```
pair<int,int> solucionDirecta(int p, int q){
    if(!esVocal(p) && esVocal(q))
        return {p,q};
    else
        return {0,0};
}
```

Si no se cumple ninguno de estos dos casos, el algoritmo continúa dividiendo el problema en subproblemas hasta llegar a los casos base. Para ello llama a la función “dividir” y se llama recursivamente:

```
int dividir(int p, int q) {
    return int((q+p)/2);
}
```

Al regresar de las llamadas recursivas llamará a la función “combinar” para conseguir la cadena más larga sacada de los subproblemas:

```
pair<int,int> combinar(pair<int,int> s1, pair<int,int> s2, int p, int q,
int m) {
```

```

if(solucionDirecta(m,m+1) == pair{0,0})
    return cadMLarga(s1,s2);

else{
    if(s1.second == m){
        if(s2.first == m+1){
            return {s1.first,s2.second};
        }
        else{
            s1 = extenderDer(s1);
            return cadMLarga(s1,s2);
        }
    }
    else{
        if(s2.first == m+1){
            s2 = extenderIzq(s2);
            return cadMLarga(s1,s2);
        }
        else{
            pair<int,int> s3 = cadFrontera(p,q,m);
            pair<int,int> s4 = cadMLarga(s1,s2);
            return cadMLarga(s3,s4);
        }
    }
}
return {0,0};
}

```

Esta función comprueba varios casos:

- En primer lugar, comprueba que no haya continuidad en la frontera, es decir, que las dos soluciones del subproblema no puedan formar una solución diferente.
- Después comprueba cómo es la continuidad:
 - Si la primera cadena solución termina en el punto en el que se corta el subproblema, comprobará:
 - Si la segunda cadena solución comienza en el punto en el que se corta el subproblema, devolviendo las dos soluciones juntas.
 - Si no, extenderá la primera lo máximo posible por la derecha y devolverá la solución más larga.
 - Si la primera no llega hasta donde se corta, comprobará:
 - Si la segunda comienza en ese punto, extenderá esta solución lo máximo posible por la izquierda y devolverá la cadena más larga.
 - Si no, significa que hay una cadena que fue cortada al dividir, por lo que tendrá que formarla y extender por ambos lados lo máximo posible. Entonces devolverá la solución más larga de entre las tres que hemos obtenido.

Si no se cumpliese ningún caso, devolvería una solución no válida.

El código completo en orden es:

```
#include <iostream>
#include <stdio.h>
#include <time.h>
#include <cstring>

#include <time.h>
#include <sys/time.h>
#include <fstream>

using namespace std;

// primer ejemplo largo usado:
//const char A[] =
{'a','b','c','c','a','b','e','a','d','d','e','d','a','d','e','b','d','a',
'e'};

const int N = 170000;
char * A;

ofstream cMejor, cPeor, cAleatorio, cTodos;
//ofstream salidaDyV, salidaSD;
const int minimo = 10000;
const int maximo = 210000;

bool esVocal (int i){
    switch (A[i]){
        case ('a'):
        case ('e'):
        case ('i'):
        case ('o'):
        case ('u'):
            return true;
            break;
        default :
            return false;
    }
}

pair<int,int> solucionDirecta(int p, int q){
    if(!esVocal(p) && esVocal(q))
        return {p,q};
    else
        return {0,0};
}

int dividir(int p, int q) {
```

```

        return int((q+p)/2);
    }

pair<int,int> cadMLarga(pair<int,int> s1, pair<int,int> s2){
    if((s1.second-s1.first) >= (s2.second-s2.first))
        return s1;
    else
        return s2;
}

pair<int,int> extenderDer(pair<int,int> s1){
    int x = s1.second;
    while((x+2 <= strlen(A)-1) && solucionDirecta(x+1,x+2) != pair{0,0})
        x += 2;
    return {s1.first,x};
}

pair<int,int> extenderIzq(pair<int,int> s2){
    int x = s2.first;
    while((x-2 >= 0) && solucionDirecta(x-2,x-1) != pair{0,0})
        x -= 2;
    return {x,s2.second};
}

pair<int,int> cadFrontera(int p, int q, int m){
    pair<int,int> s3 = {m,m+1};
    s3 = extenderIzq(s3);
    s3 = extenderDer(s3);
    return s3;
}

pair<int,int> combinar(pair<int,int> s1, pair<int,int> s2, int p, int q,
int m) {
    if(solucionDirecta(m,m+1) == pair{0,0})
        return cadMLarga(s1,s2);

    else{
        if(s1.second == m){
            if(s2.first == m+1){
                return {s1.first,s2.second};
            }
            else{
                s1 = extenderDer(s1);
                return cadMLarga(s1,s2);
            }
        }
        else{
            if(s2.first == m+1){
                s2 = extenderIzq(s2);

```

```

        return cadMLarga(s1,s2);
    }
    else{
        pair<int,int> s3 = cadFrontera(p,q,m);
        pair<int,int> s4 = cadMLarga(s1,s2);
        return cadMLarga(s3,s4);
    }
}
}
return {0,0};
}

pair<int,int> DyV(int p, int q) {
    if (p == q)
        return {0,0};
    else if (q - p == 1)
        return solucionDirecta(p,q);
    else {
        int m = dividir(p,q);
        pair<int,int> s1 = DyV(p,m);
        pair<int,int> s2 = DyV(m+1,q);
        return combinar(s1,s2,p,q,m);
    }
}

char * generadorEntrada(int n) {
    char * cadena = new char[n];
    for(int i = 0; i < n; i++) {
        cadena[i] = (char) (97 + rand() % (122 - 97 + 1));
    }
    return cadena;
}

// todo consonante-vocal
char * generadorEntradaPeor(int n)
{
    char * cadena = new char[n];
    for(int i = 0; i < n; i++)
    {
        if(i%2 == 0){
            cadena[i] = 'b';
        }
        else{
            cadena[i] = 'a';
        }
    }

    return cadena;
}

```



```

}

// que nunca se cumpla el patrón
char * generadorEntradaMejor(int n)
{
    char * cadena = new char[n];
    for(int i = 0; i < n; i++)
    {
        cadena[i] = 'a';
    }

    return cadena;
}

void imprimirSol (pair<int,int> solucion){
    if(solucion == pair{0,0})
        cout << "No hay cadenas consonante-vocal de ninguna longitud" <<
endl;
    else{
        cout << "La solución es ";
        for(int i = solucion.first; i <= solucion.second; i++){
            cout << A[i];
        }
        cout << " en la posición " << solucion.first + 1 << endl;
    }
}

void imprimirArray (){
    for(int i = 0; i < strlen(A); i++){
        cout << A[i];
    }
    cout << endl;
}

string obtenerTiempo(int p, int q) {
    struct timeval t1, t2;
    double tiempo;
    gettimeofday(&t1, NULL);
    pair<int, int> solucion = DyV(p,q);
    gettimeofday(&t2, NULL);
    tiempo = (t2.tv_sec - t1.tv_sec)*1000 + (t2.tv_usec -
t1.tv_usec)/1000.0;
    string t = "";
    string cadenaTiempo = to_string(tiempo);
    for(int h = 0; h < cadenaTiempo.length(); h++)
    {
        if(cadenaTiempo[h] != '.')
            t = t + cadenaTiempo[h];
        else t = t + ',';
    }
}

```

```

    }
    return t;
}

void archivosCSV()
{
    cTodos.open("todos.csv");
    cAleatorio.open("aleatorio.csv");
    cPeor.open("peor.csv");
    cMejor.open("mejor.csv");

    int p,q,m;
    cMejor << "n;tm(n)" << endl;
    cPeor << "n;tM(n)" << endl;
    cTodos << "n;tM(n);tm(n)" << endl;
    for(int n = minimo; n <= maximo; n+=10000)
    {
        // generacion de la entrada aleatoria para una n dada
        p = 0;
        q = n - 1;
        A = generadorEntradaPeor(n);
        string tM = obtenerTiempo(p,q);
        A = generadorEntradaMejor(n);
        string tm = obtenerTiempo(p,q);

        m = n/1000;

        // añade al fichero csv la n usada y los milisegundos que ha
        // tardado el algoritmo (diferencia entre t1 y t2)
        cTodos << n << ";" << tM << ";" << tm << endl;
        cMejor << n << ";" << tm << endl;
        cPeor << n << ";" << tM << endl;

    }

}

int main(){

    srand(time(NULL));

    // llama al generador de casos
    // lee el fichero y lo va metiendo en un array de caracteres a la vez
    // que cuenta

```

```

    cout << "¿Qué desea hacer? (introduzca el número de la opción)" <<
endl;
    cout << "1 - Analizar un número a indicar de arrays" << endl;
    cout << "2 - Producir datos sobre el tiempo que se guardarán en un
archivo .csv" << endl;
    cout << "3 - Ayuda" << endl;

    int opcion;
    cin >> opcion;
    if (opcion == 1){
        int c;
        cout << "Introduzca el número de cadenas que quiere analizar" <<
endl;
        cin >> c;
        int array;
        cout << "¿Quiere saber cuáles son los arrays? (1 = SI; 0 = NO)"
<< endl;
        cin >> array;
        for (int i = 0; i < c; i++){
            cout << i+1 << "-";
            A = generadorEntrada(N);
            if (array == 1){
                imprimirArray();
            }
            int p = 0;
            int q = strlen(A)-1;
            pair<int,int> solucion = DyV(p,q);
            imprimirSol(solucion);
        }
    }
    else if(opcion == 2) {
        archivosCSV();
    }
    else {
        cout << "1 - Se generará de manera aleatoria un array de
caracteres (alfabeto únicamente) que se analizará a través de un
algoritmo basado en \"Divide y Vencerás\" para comprobar cuál es la
subcadena más larga que cumpla con el patrón consonante-vocal. Usted
determinará el número de cadenas a analizar." << endl;
        cout << "2 - Se hará un estudio del tiempo sobre dos tipos de
entradas: el peor caso y el mejor. Insertará los datos en archivos .csv,
es decir, en hojas de cálculo." << endl;
    }

    return 0;
}

```

El problema lo hemos resuelto de esta manera debido a que, una vez divides el problema original en subproblemas, para poder llegar a la solución más larga hay que ir uniendo estos. Existe la posibilidad de que, al dividir, se haya separado una solución, por lo que hay que ir comprobando si es el caso y conseguir la mejor solución.

5. Validación del algoritmo

Una vez completado el código comenzamos a probar con arrays pequeños como:

- {a,b}: No hay cadenas consonante-vocal de ninguna longitud
- {b,a}: La solución es ba en la posición 1
- {a,b,a,b}: La solución es ba en la posición 2
- {b,a,b,a}: La solución es baba en la posición 1

Cuando comprobamos que daba la solución correcta probamos con un caso más largo:

- {a,b,c,c,a,b,e,a,d,d,e,d,a,d,e,b,d,a,e}: La solución es dedade en la posición 10
- {b,c,e,a,b,a,b,e,k,s,s,a,b,a,b,z,y,x}: La solución es babe en la posición 5
- {a,b,c,e,h,f,e,k,s,r,t,z,y,x,d,b,c,e,h,a,e,k,s,r,t,z,y,x}: La solución es ceha en la posición 17
- {a,o,a,e,u,c,b,d,s,b,p,k,l,b,b,c,c,s,b,z,x,z,z,y,x}: No hay cadenas consonante-vocal de ninguna longitud

Como esto se realizó de manera correcta, pasamos a generar arrays aleatorios con un tamaño mayor. Algunos de estos son:

- {f,d,i,p,e,y,p,p,u,r,u,m,u,b,b,c,n,s,g,e,y,w,y,h,t,e,y,e,t,p,q,z,u,y,q,z,y,f,o,s,z,i,f,t,j,g,y,w,a,g}: La solución es purumu en la posición 8
- {d,a,c,d,h,w,j,i,a,c,x,s,d,r,q,v,q,p,d,e,h,c,o,o,x,x,x,v,w,x,b,b,y,e,e,h,c,n,r,e,r,o,w,x,g,m,s,y,d,x}: La solución es rero en la posición 39
- {f,n,z,t,d,x,t,a,u,r,a,y,s,a,e,y,j,g,n,b,k,g,p,i,d,x,w,y,y,a,v,d,p,x,y,s,w,r,v,s,k,v,q,e,v,u,c,e,c,p}: La solución es qevuce en la posición 43
- {y,m,u,w,j,q,r,f,a,l,x,i,j,w,u,i,j,q,y,j,d,d,b,l,c,h,k,x,q,c,l,q,p,f,o,a,y,f,h,y,r,e,i,c,c,c,k,l,u,i}: La solución es mu en la posición 2
- {b,h,c,z,m,j,r,n,s,p,r,s,s,m,r,v,d,r,q,p,u,h,y,x,w,t,v,h,u,q,i,x,z,k,x,l,t,q,a,n,f,s,g,z,e,z,u,j,q,m}: La solución es zezu en la posición 44

6. Estudio experimental del tiempo de ejecución

Para hacer el estudio experimental del tiempo de ejecución hemos implementado en el código varias funciones:

- Un generador de casos aleatorios. Genera de manera aleatoria las cadenas a analizar. Será el que genere las pruebas de forma habitual, insertando el número de cadenas que se quieren analizar.

```
char * generadorEntrada(int n) {
    char * cadena = new char[n];
    for(int i = 0; i < n; i++) {
        cadena[i] = (char) (97 + rand() % (122 - 97 + 1));
    }
    return cadena;
}
```

- Un generador del caso peor. Genera el peor caso posible de entrada, ya que será una entrada que tarde mucho en ejecutar. En este caso se trata de un array de tamaño n que cumpla entero el patrón, por lo que la solución sería de tamaño n y tendría que ejecutar la parte más costosa de combinar un gran número de veces.

```
char * generadorEntradaPeor(int n)
{
    char * cadena = new char[n];
    for(int i = 0; i < n; i++)
    {
        if(i%2 == 0){
            cadena[i] = 'b';
        }
        else{
            cadena[i] = 'a';
        }
    }

    return cadena;
}
```

- Un generador del caso mejor. Genera el mejor caso posible de entrada, ya que será una entrada que tarde poco en ejecutar. En este caso se trata de un array de tamaño n que nunca cumpla el patrón, por lo que no habría una solución válida y se quedaría en el primer if de la función combinar.

```
char * generadorEntradaMejor(int n)
{
    char * cadena = new char[n];
    for(int i = 0; i < n; i++)
    {
        cadena[i] = 'a';
    }

    return cadena;
}
```

- ```
}
```
- Una función que calcula el tiempo que tarda en ejecutarse el programa:
  - Una función que genera ficheros con extensión csv, es decir, hojas de cálculo en las que almacenamos los datos obtenidos y a partir de las que podemos obtener gráficas.

```
void archivosCSV()
{
 cTodos.open("todos.csv");
 cAleatorio.open("aleatorio.csv");
 cPeor.open("peor.csv");
 cMejor.open("mejor.csv");

 int p,q,m;
 cMejor << "n;tm(n)" << endl;
 cPeor << "n;tM(n)" << endl;
 cAleatorio << "n;t" << endl;
 cTodos << "n;tM(n);tm(n);t(n)" << endl;
 for(int n = minimo; n <= maximo; n+=10000)
 {
 p = 0;
 q = n - 1;
 cout << "Caso peor: ";
 A = generadorEntradaPeor(n);
 string tM = obtenerTiempo(p,q);
 cout << endl;
 cout << "Caso mejor: ";
 A = generadorEntradaMejor(n);
 string tm = obtenerTiempo(p,q);
 cout << endl;
 cout << "Caso aleatorio: ";
 A = generadorEntrada(n);
 string t = obtenerTiempo(p,q);
 cout << endl;

 m = n/1000;

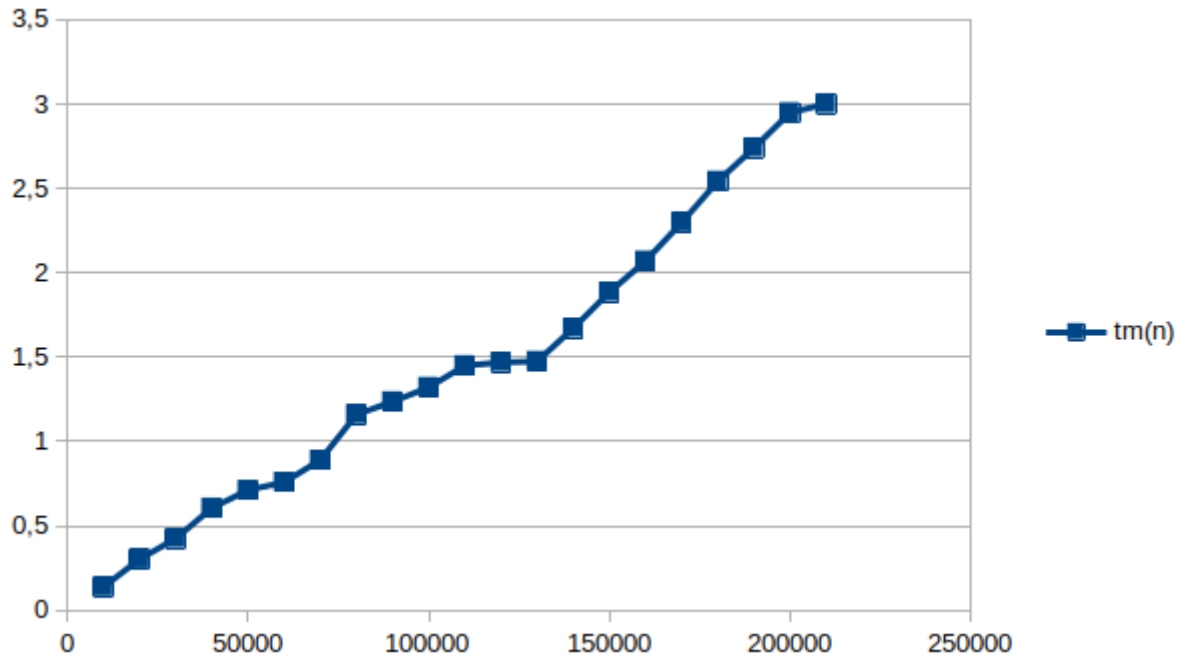
 cTodos << n << ";" << tM << ";" << tm << ";" << t << endl;
 cMejor << n << ";" << tm << endl;
 cPeor << n << ";" << tM << endl;
 cAleatorio << n << ";" << t << endl;

 }
}
```

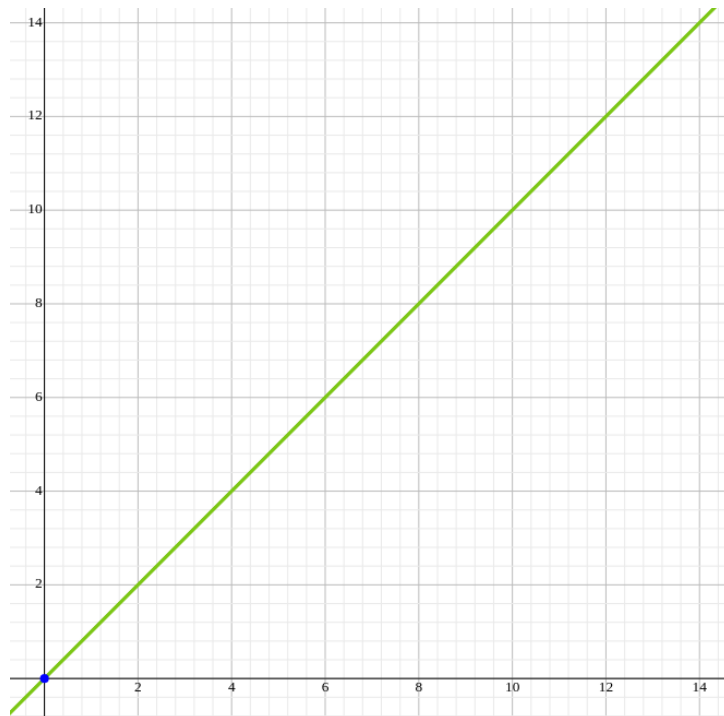
## 7. Contraste del estudio teórico y el experimental

Vamos a comparar las gráficas resultantes del estudio teórico y del experimental:

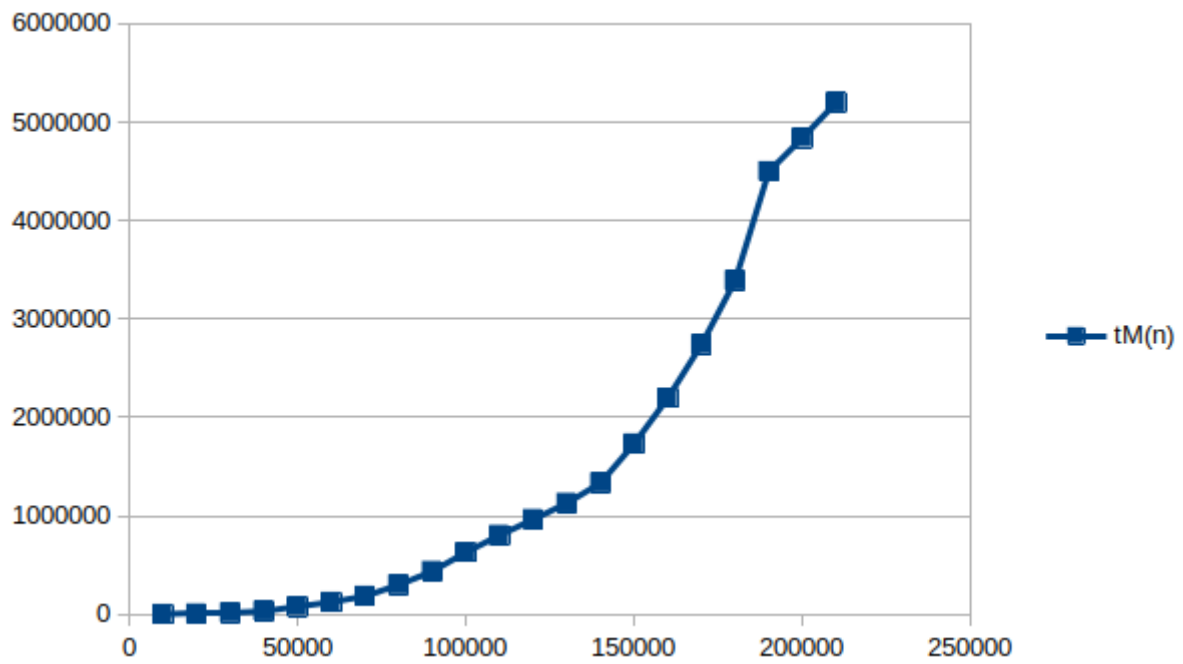
- Estudio mejor (experimental):



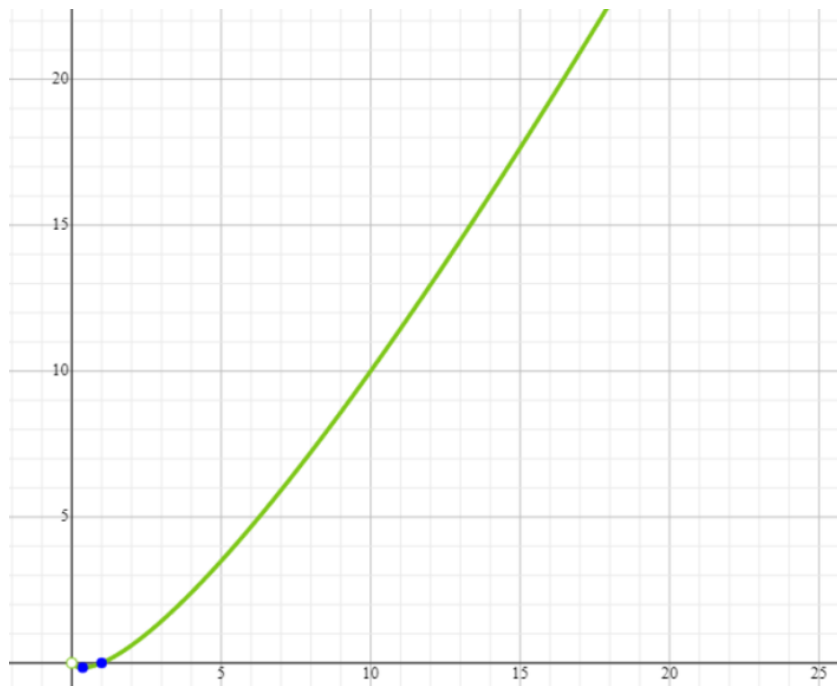
- Estudio mejor (teórico),  $tm(n)$ :



-Estudio peor (experimental):



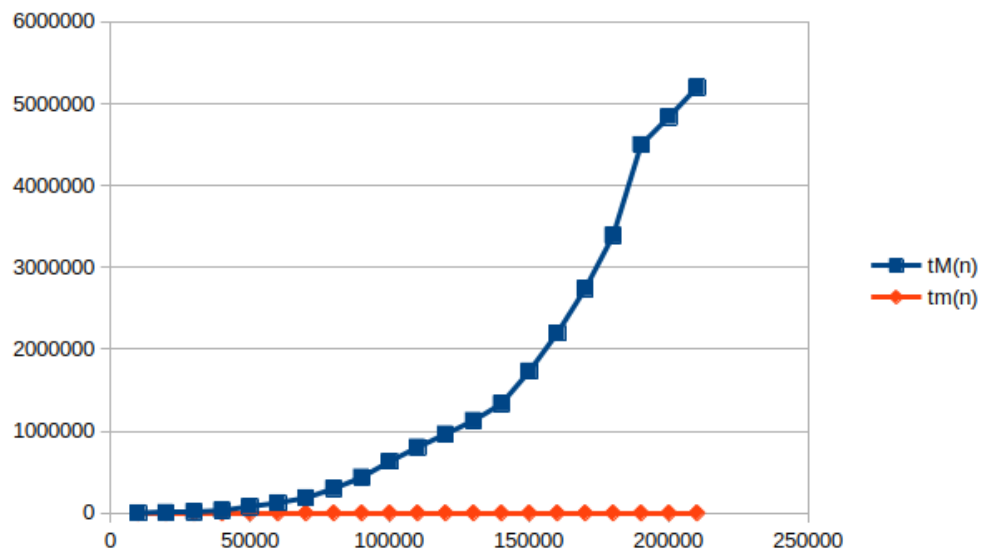
- Estudio peor (teórico),  $tM(n \log n)$ :



Podemos observar que ambos estudios del peor y el mejor caso progresan de forma similar a lo esperado.



- Estudio conjunto (peor y mejor):



Para las gráficas del estudio teórica hemos usado [symbolab](#).

## 8. Conclusión

Este trabajo ha sido bastante rápido de hacer y muy sencillo. Tras hacer este ejercicio y gracias a los mínimos problemas que hemos tenido, hemos observado cómo el algoritmo de "divide y vencerás" nos permite abordar problemas complejos de manera estructurada y eficiente. Además de ayudarnos a identificar oportunidades para dividir el problema en subproblemas más pequeños para garantizar la resolución correcta de cada subproblema y finalmente combinar las soluciones de manera coherente y precisa.

Estamos muy orgullosas de nuestro trabajo y de las soluciones obtenidas en muy poco tiempo.